



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

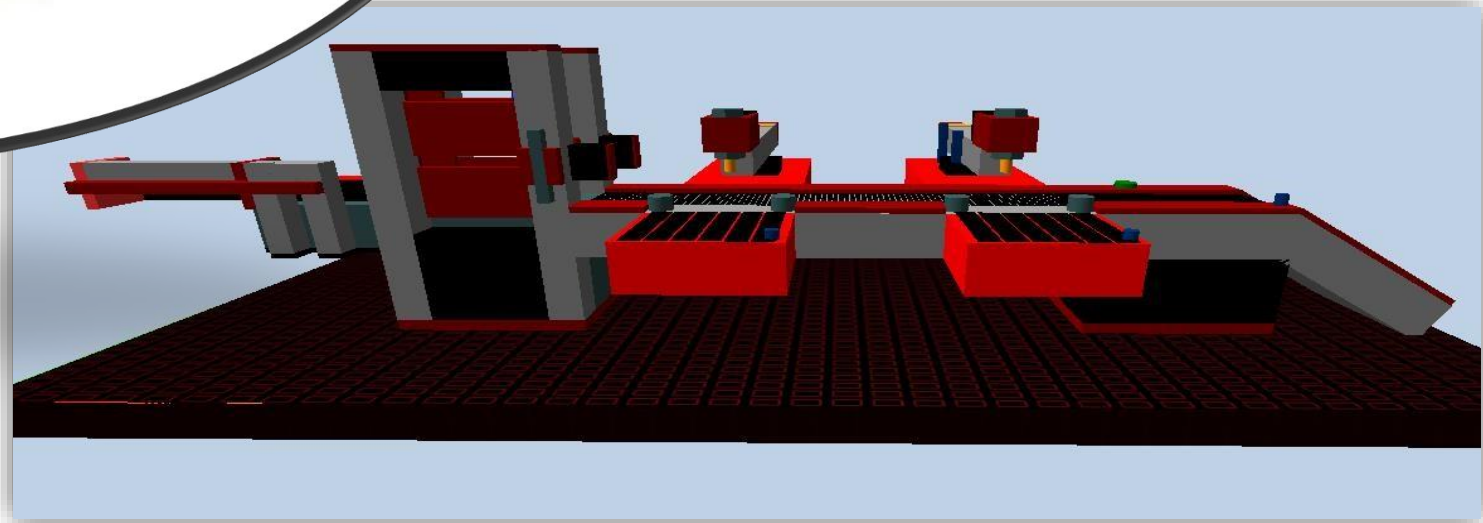
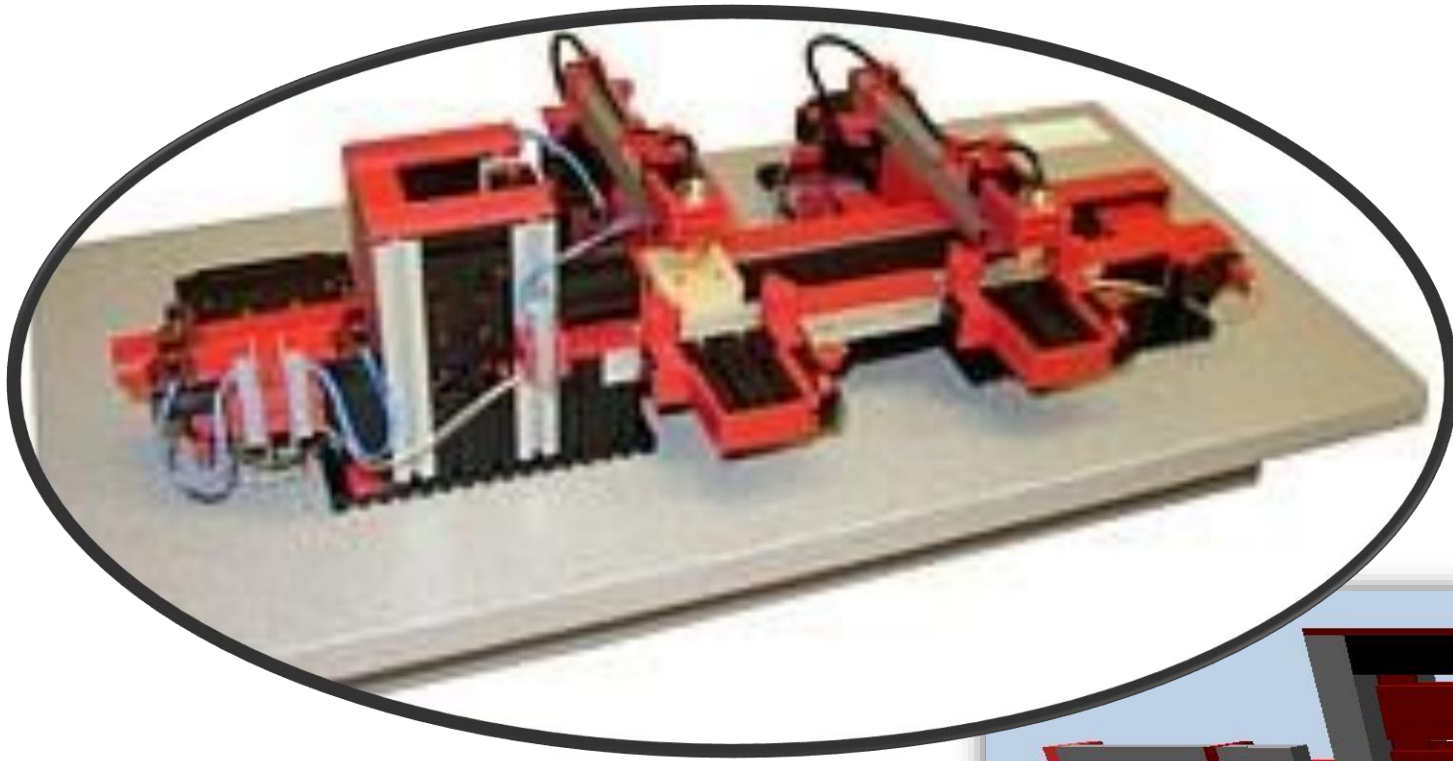
STR

LAB #7

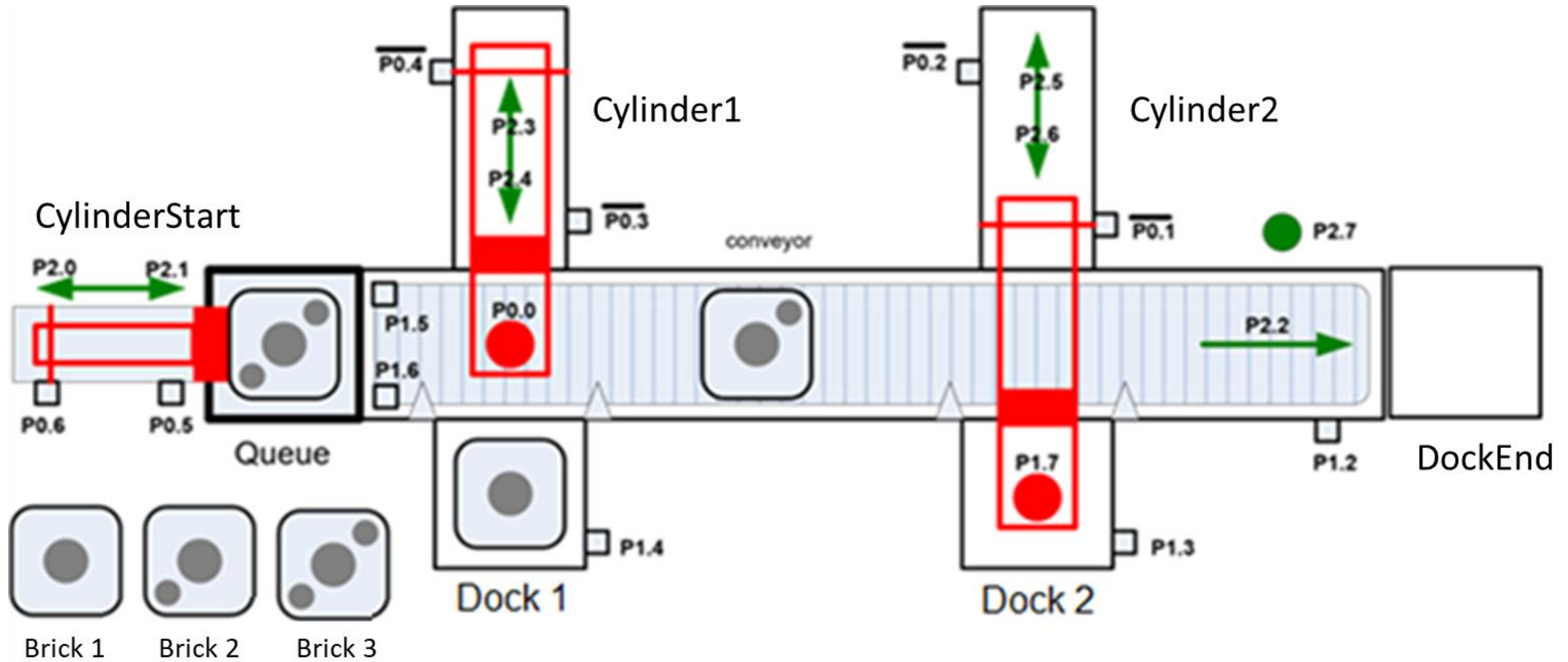
LABWORK 2:

JNI development for interacting with the
physical system

2022 - 2023



LABWORK 2



PART 1: JNI development

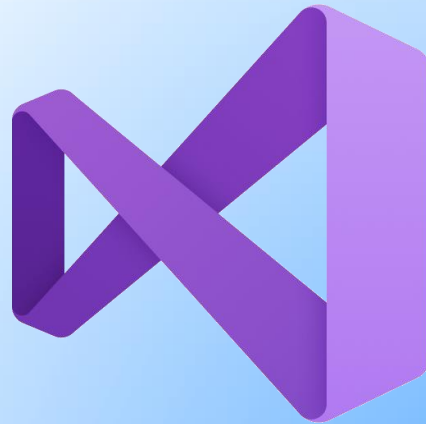
- ☐ [Creation of the JNI interface](#)
- ☐ [Accessing low-level functionality](#)
- ☐ [Testing from Java Application](#)

PART 2: Java Real-Time

- ☐ [The Cylinder Interface](#)
- ☐ [Developing class Mechanism](#)
- ☐ [Identification of needed Java Threads](#)

PART 1

JNI development for interacting with the physical system



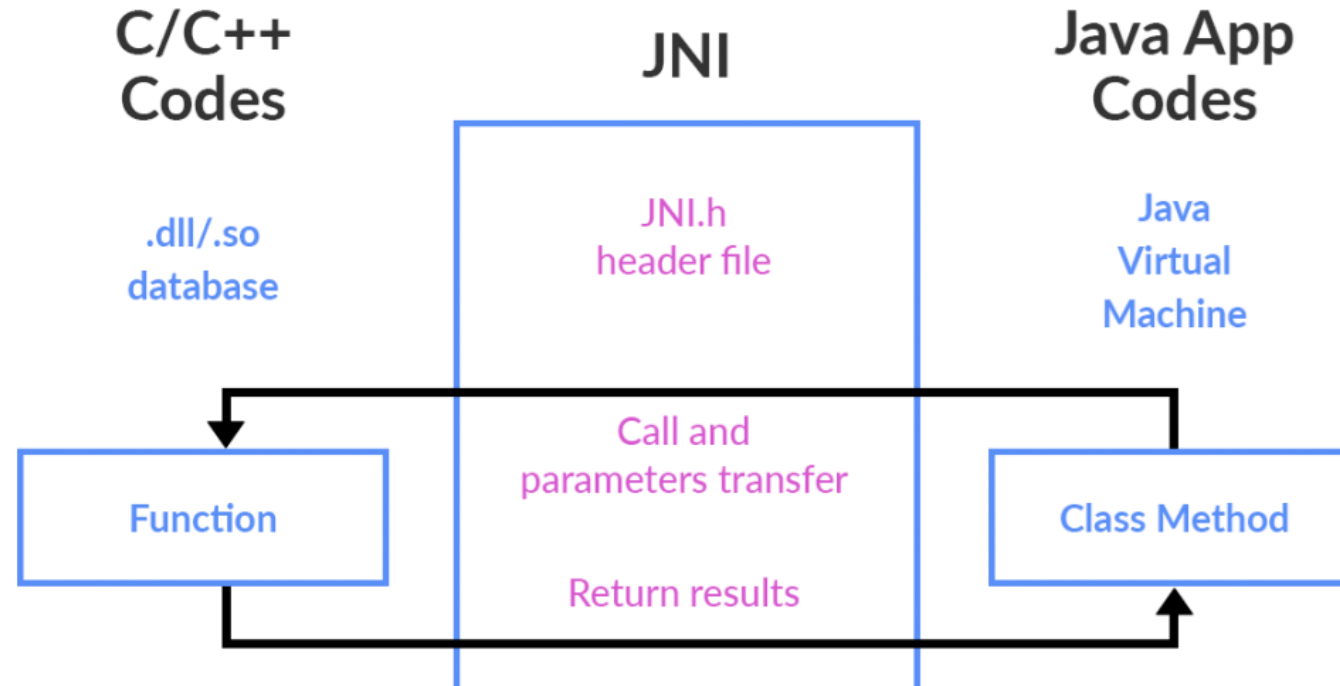
Creation of the JNI interface



We are now ready to proceed to the next phase, JNI interoperability, for utilization within the Java Application

While you can write applications entirely in Java, there are situations where Java alone does not meet the needs of your application. Programmers use the **JNI** to write Java native methods to handle those situations when an application cannot be written entirely in Java.

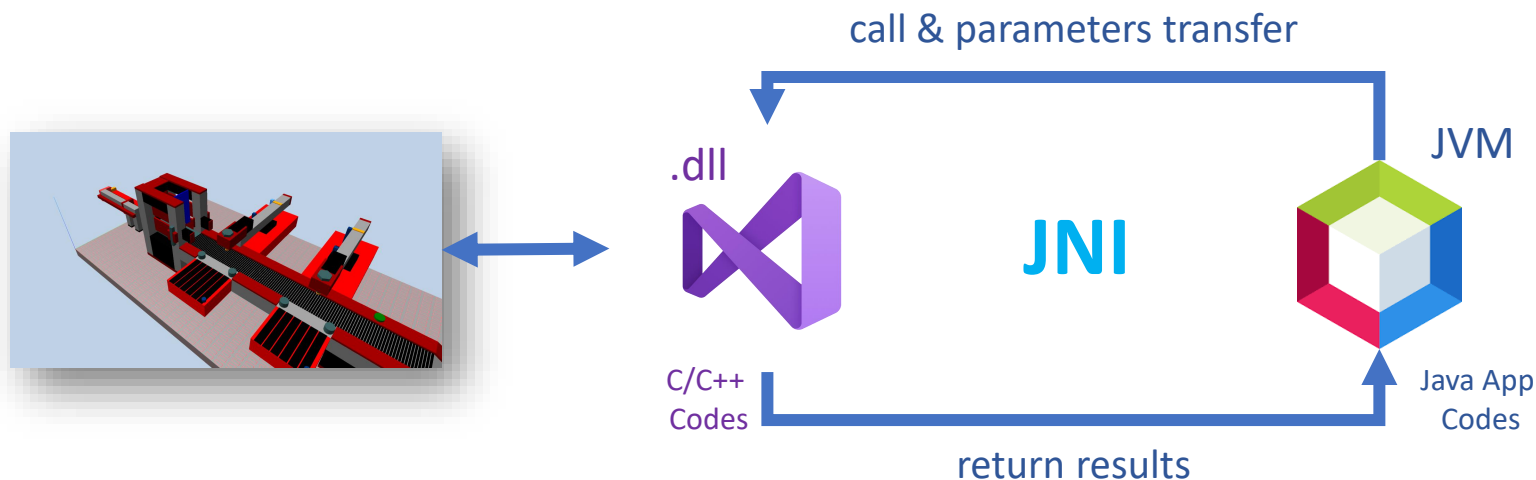
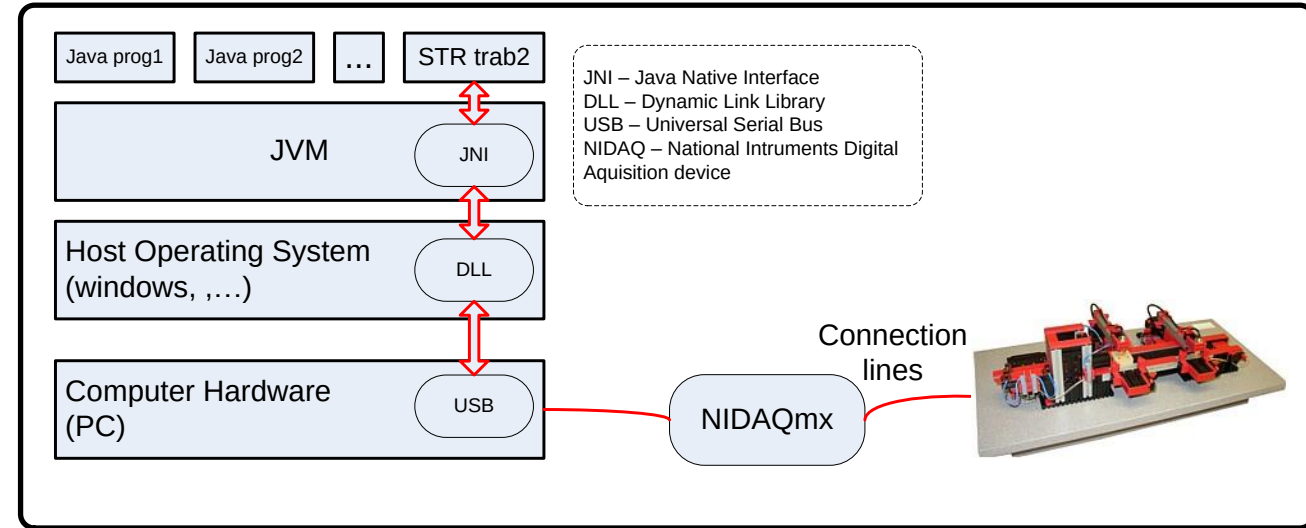
<https://docs.oracle.com/javase/7/docs/technotes/guides/jni/spec/intro.html>



Creation of the JNI interface

Java Application

- Create a **Java Application** using the **NetBeans** environment
- Proceed with the development of the JNI mechanism
- Create a demo **class** to verify that the Java Program can interact with the Splitter Conveyor



Creation of the JNI interface

Signatures in Java naming convention (complete table):

Sensors signature functions in C	In Java
int cylinder1_getPosition()	native public int cylinder1GetPosition();
int getIdentificationSensors()	native public int getIdentificationSensors();
bool isBrickAtDock1()	native public boolean isBrickAtDock1 ();
(complete the table)	

Actuator signature functions in C	In Java
void initializeHardwarePorts();	native public void initializeHardwarePorts();
void cylinder1_moveForward()	native public void cylinder1MoveForward();
void cylinder1_moveBackward()	native public void cylinder1MoveBackward();
void cylinder1_stop()	native public void cylinder1Stop ();
void conveyorMove()	native public void conveyorMove();
void conveyorStop()	native public void conveyorStop();
(complete the table)	



Copy the Java signatures from the tables into a new class **SplitterConveyor** (see next slide).

Creation of the JNI interface

Create class SplitterConveyor



The screenshot shows the Apache NetBeans IDE 12.5 interface. The 'New' menu is open, and the 'New Java Class' dialog is displayed. The 'Name and Location' section is highlighted with a red box, showing the following fields:

- Class Name: SplitterConveyor
- Project: labwork2
- Location: Source Packages
- Package: (empty)
- Created File: C:\STR\labwork2\src\SplitterConveyor.java

The 'Finish' button at the bottom right of the dialog is also highlighted with a red box. A warning message at the bottom of the dialog states: "Warning: It is highly recommended that you do not place Java classes in the default package."

Creation of the JNI interface

The SplitterConveyor class

Copy the signatures. Ensure file is saved.



```
9  /*
10  public class SplitterConveyor {
11      static synchronized native public void initializeHardwarePorts();
12      static synchronized native public void cylinder1MoveForward();
13      static synchronized native public void cylinder1MoveBackward();
14      static synchronized native public void cylinder1stop();
15      static synchronized native public void conveyorMove();
16      static synchronized native public void conveyorStop();
17
18      static synchronized native public int cylinder1GetPosition();
19      static synchronized native public int getIdentificationSensors();
20      static synchronized native public boolean isBrickAtDock1();
21
22  }
```

Creation of the JNI interface

Generate heading file



Launch a console cmd.exe

Considering the working directory (C:\STR), execute the command:

```
cd\str\labwork2\src
```

Then execute:

```
javac SplitterConveyor.java
javah SplitterConveyor
dir
del *.class
```

As you can see, the file **SplitterConveyor.h** has been created, which needs to be included into the Visual Studio Project.

```
Command Prompt
Microsoft Windows [Version 10.0.18363.1556]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ain>cd\str\labwork2\src
C:\STR\labwork2\src>javac SplitterConveyor.java
C:\STR\labwork2\src>javah -cp . SplitterConveyor
C:\STR\labwork2\src>dir
Volume in drive C is Windows
Volume Serial Number is 86D1-E9C6

Directory of C:\STR\labwork2\src

14-Nov-21  02:00 AM    <DIR>          .
14-Nov-21  02:00 AM    <DIR>          ..
11-Nov-21  04:11 PM                371 Labwork2.java
14-Nov-21  02:00 AM                475 SplitterConveyor.class
14-Nov-21  01:59 AM            1,921 SplitterConveyor.h
14-Nov-21  01:52 AM                874 SplitterConveyor.java
               4 File(s)              3,641 bytes
               2 Dir(s) 23,775,948,800 bytes free

C:\STR\labwork2\src>del *.class
C:\STR\labwork2\src>
```

Creation of the JNI interface

The **obtained** signatures



The screenshot shows an IDE with the following components:

- Projects:** A tree view on the left showing the project structure: `labwork2` (Source Packages) containing `<default package>`, `Labwork2.java`, `SplitterConveyor.h`, and `SplitterConveyor.java`. Below this are `Test Packages`, `Libraries`, and `Test Libraries`.
- Navigator:** A panel below the Projects view showing the `SplitterConveyor` class with its members: `SplitterConveyor()`, `conveyorMove()`, `conveyorStop()`, `cylinder1MoveBackward()`, `cylinder1MoveForward()`, `cylinder1Stop()`, `getCylinder1Position() : int`, `getIdentificationSensors() : int`, `initializeHardwarePorts()`, and `isPartAtDock1() : boolean`.
- Editor:** The main window showing the `SplitterConveyor.h` file. It contains the following JNI interface definitions:

```
12  /*  
13   * Signature: ()V  
14   */  
15  JNIEXPORT void JNICALL Java_SplitterConveyor_initializeHardwarePorts  
16      (JNIEnv *, jclass);  
17  
18  /*  
19   * Class:      SplitterConveyor  
20   * Method:     cylinder1MoveForward  
21   * Signature:  ()V  
22   */  
23  JNIEXPORT void JNICALL Java_SplitterConveyor_cylinder1MoveForward  
24      (JNIEnv *, jclass);  
25  
26  /*  
27   * Class:      SplitterConveyor  
28   * Method:     cylinder1MoveBackward  
29   * Signature:  ()V  
30   */  
31  JNIEXPORT void JNICALL Java_SplitterConveyor_cylinder1MoveBackward  
32      (JNIEnv *, jclass);  
33  
34  /*  
35   * Class:      SplitterConveyor  
36   * Method:     cylinder1Stop  
37   * Signature:  ()V  
38   */  
39  JNIEXPORT void JNICALL Java_SplitterConveyor_cylinder1Stop  
40      (JNIEnv *, jclass);  
41  
42  /*  
43   * Class:      SplitterConveyor  
44   * Method:     conveyorMove  
45   * Signature:  ()V
```
- Output:** A panel at the bottom showing the output of the `labwork2` run.

Creation of the JNI interface



In Visual Studio Project **SplitterConveyor** add the Java include path into the Visual Studio project properties.

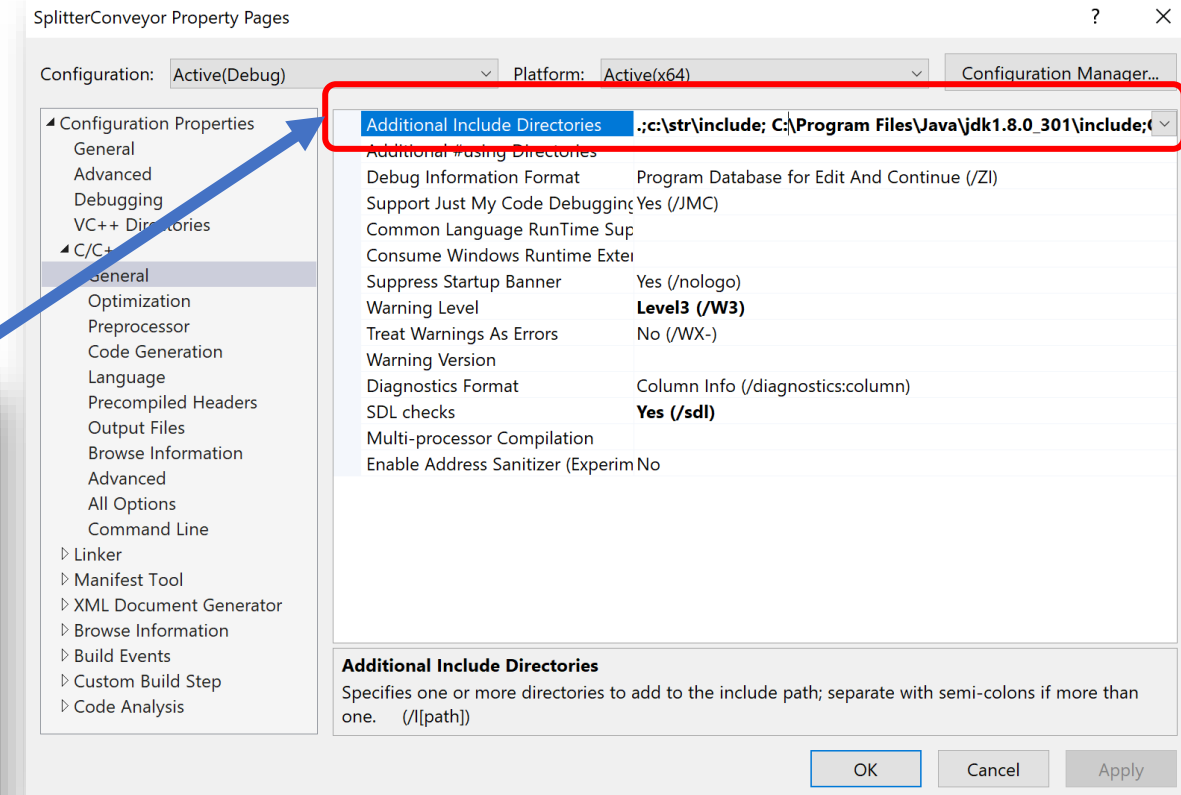
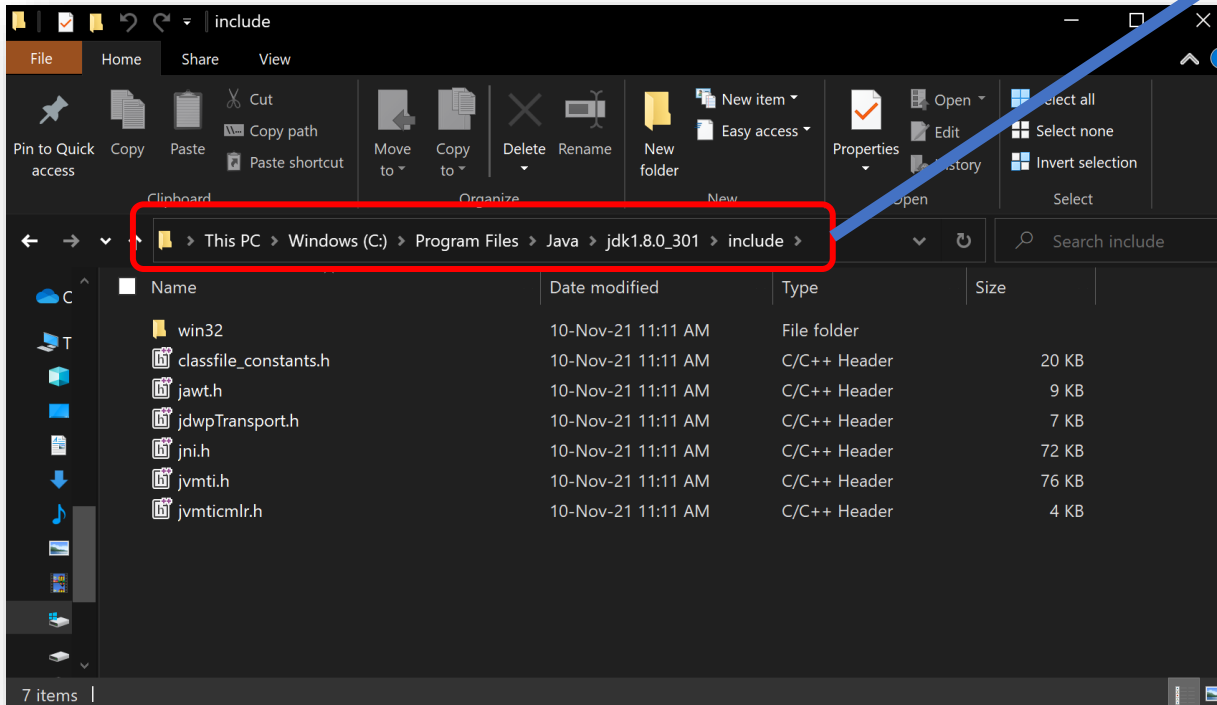


Must be something like this e.g. (depends on java version and installed path:

C:\Program Files\Java\jdk1.8.0_341\include\

C:\Program Files\Java\jdk1.8.0_341\include\win32

Split each path with a semi-colon (;)



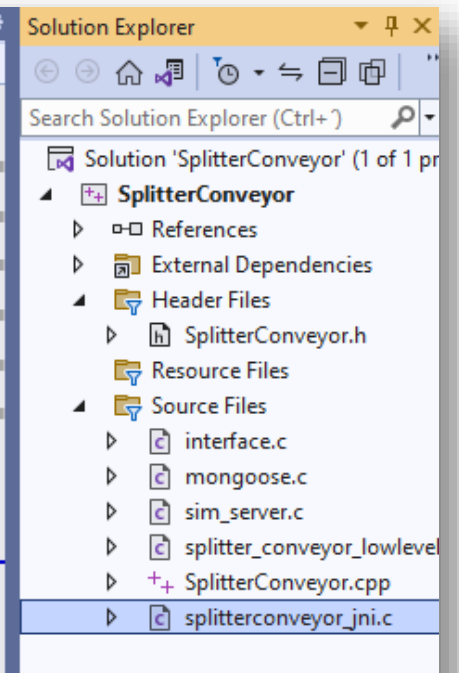
Creation of the JNI interface



Add new file `splitterconveyor_jni.c` into your project



```
1  #include<jni.h>
2  #include<SplitterConveyor.h>
3
4  JNIEXPORT void JNICALL Java_SplitterConveyor_initializeHardwarePorts(JNIEnv* ignoreEnv, jclass ignorejClass) {
5      initializeHardwarePorts();
6  }
7
8  JNIEXPORT void JNICALL Java_SplitterConveyor_cylinder1MoveForward(JNIEnv* ignoreEnv, jclass ignorejClass) {
9      cylinder1_moveForward();
10 }
11
12 JNIEXPORT void JNICALL Java_SplitterConveyor_cylinder1MoveBackward(JNIEnv* ignoreEnv, jclass ignorejClass) {
13     cylinder1_moveBackward();
14 }
15
16 JNIEXPORT void JNICALL Java_SplitterConveyor_cylinder1stop(JNIEnv* ignoreEnv, jclass ignorejClass) {
17     cylinder1_stop();
18 }
19
20 JNIEXPORT void JNICALL Java_SplitterConveyor_conveyorMove(JNIEnv* ignoreEnv, jclass ignorejClass) {
21     conveyorMove();
22 }
23
24 JNIEXPORT void JNICALL Java_SplitterConveyor_conveyorStop(JNIEnv* ignoreEnv, jclass ignorejClass) {
25     conveyorStop();
26 }
27
28 JNIEXPORT jint JNICALL Java_SplitterConveyor_cylinder1GetPosition(JNIEnv* ignoreEnv, jclass ignorejClass) {
29     return cylinder1_getPosition();
30 }
31
32 JNIEXPORT jint JNICALL Java_SplitterConveyor_getIdentificationSensors(JNIEnv* ignoreEnv, jclass ignorejClass) {
33     return getIdentificationSensors();
34 }
35
36 JNIEXPORT jboolean JNICALL Java_SplitterConveyor_isBrickAtDock1(JNIEnv* ignoreEnv, jclass ignorejClass) {
37     return isBrickAtDock1();
38 }
```



Solution Explorer shows the project structure for 'SplitterConveyor'. The files listed are: References, External Dependencies, Header Files (SplitterConveyor.h), Resource Files, and Source Files (interface.c, mongoose.c, sim_server.c, splitter_conveyor_lowlevel.c, SplitterConveyor.cpp, and splitterconveyor_jni.c).

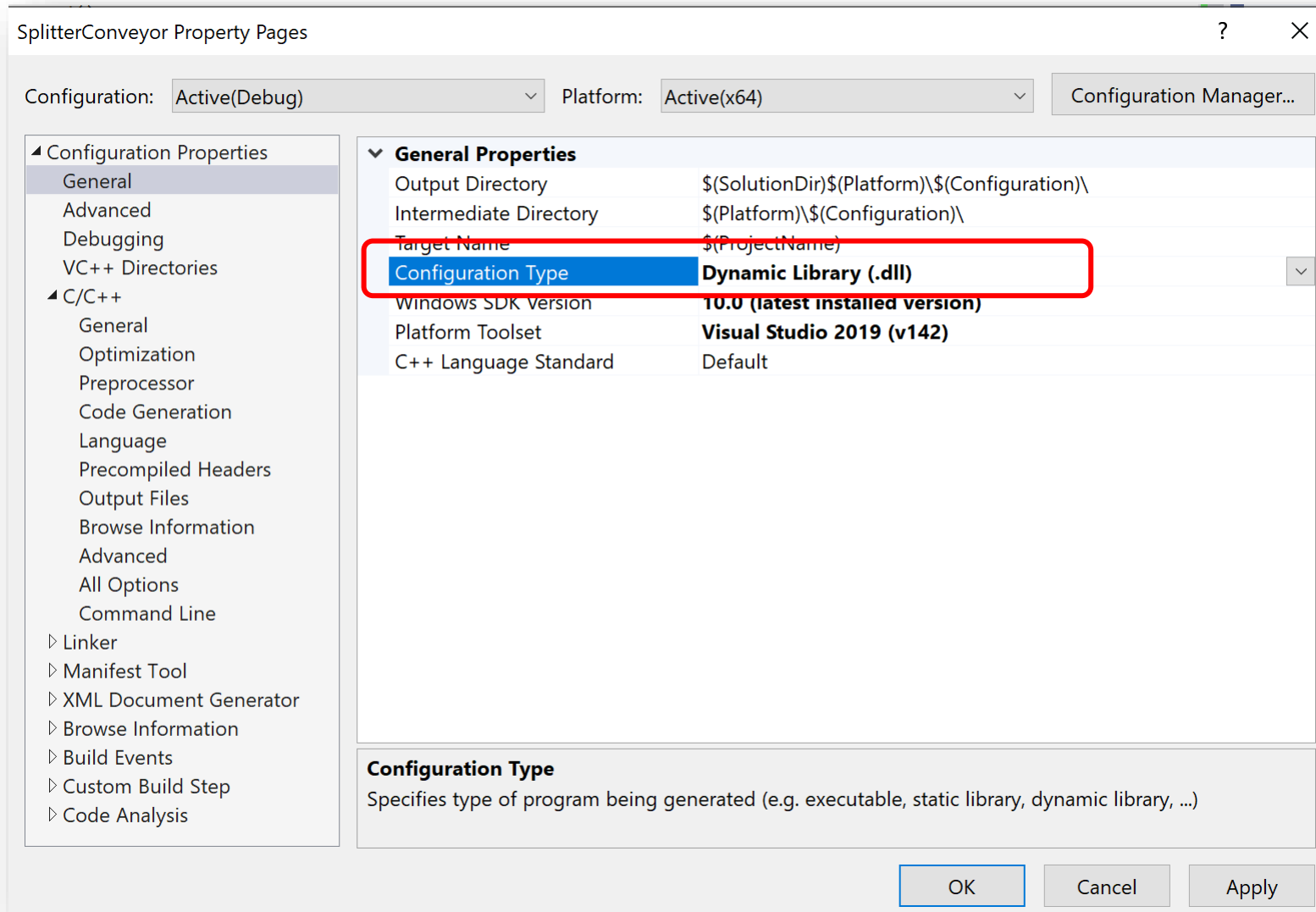
Complete as illustrated in the picture for all the functions

Functions called from the JNI stubs are specified in `SplitterConveyor.h` and developed in `splitter_conveyor_lowlevel.c`

Creation of the JNI interface



Transform Visual Studio project into **DLL**



Creation of the JNI interface



Build the project



Build the project and...



Hint: **don't try run it** (not an executable *.exe anymore)

```
Output
Show output from: Build
1>Generating Code...
1>SplitterConveyor.cpp
1> Creating library C:\STR\SplitterConveyor\x64\Debug\SplitterConveyor.lib and object C:\STR\SplitterConveyor\x64\Debug\SplitterConveyor.exp
1>SplitterConveyor.vcxproj -> C:\STR\SplitterConveyor\x64\Debug\SplitterConveyor.dll
1>Done building project "SplitterConveyor.vcxproj".
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

After that, copy the path for the generated DLL, as shown in the figure.

Creation of the JNI interface

Code to load DLL from Java



Copy-past the DLL path inside SplitterConveyor.java class

Hint: Write the empty string ("") first, then copy-past. This way, double backslashes are added automatically.

```
public class SplitterConveyor {  
  
    static{  
        System.load( filename:"C:\\str\\SplitterConveyor\\x64\\Debug\\SplitterConveyor.dll");  
    }  
  
    // these methods have the race condition effect  
    static synchronized native public void initializeHardwarePorts();  
    static synchronized native public void cylinder1MoveForward();  
    static synchronized native public void cylinder1MoveBackward();  
    static synchronized native public void cylinder1stop();  
    static synchronized native public void conveyorMove();  
    static synchronized native public void conveyorStop();  
    // (...)  
  
    // these methods just read sensors... so no race condition "problem"  
    static synchronized native public int cylinder1GetPosition();  
    static synchronized native public int getIdentificationSensors();  
    static synchronized native public boolean isBrickAtDock1();  
    // (...)  
}
```

Accessing low-level functionality

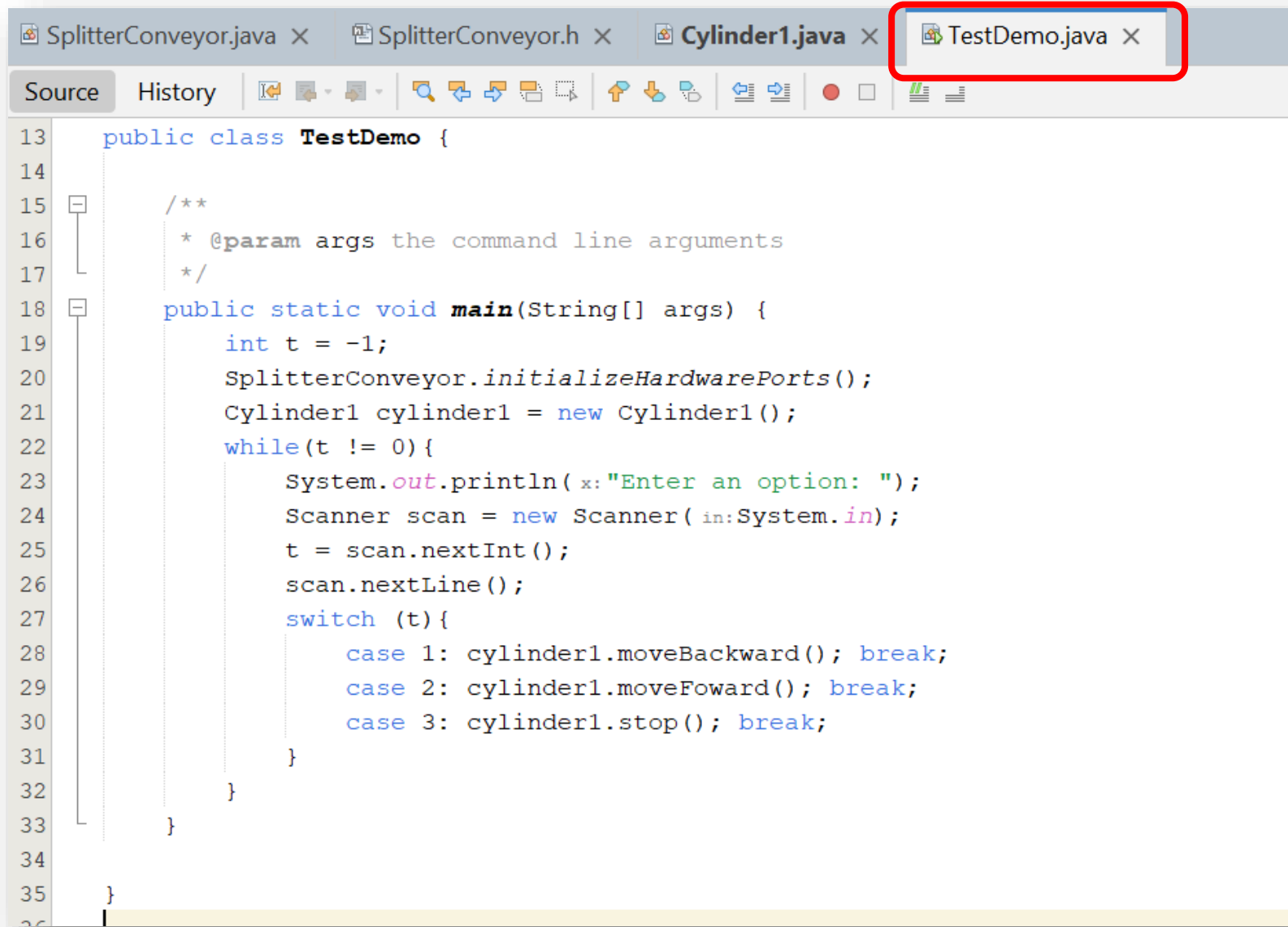
Add new Java class (1): Add a new class named **Cylinder1.java**



```
10 public class Cylinder1 {
11
12     public void moveFoward() {
13         SplitterConveyor.cylinder1MoveForward();
14     }
15     public void moveBackward() {
16         SplitterConveyor.cylinder1MoveBackward();
17     }
18     public void stop() {
19         SplitterConveyor.cylinder1stop();
20     }
21     public int cylinder1GetPosition() {
22         return SplitterConveyor.cylinder1GetPosition();
23     }
24     public void gotoPosition(int position) {
25         //TODO: move from 0 to 1 and vice-versa
26         // this part is developed in JAVA
27     }
28 }
29
```

Testing from Java Application

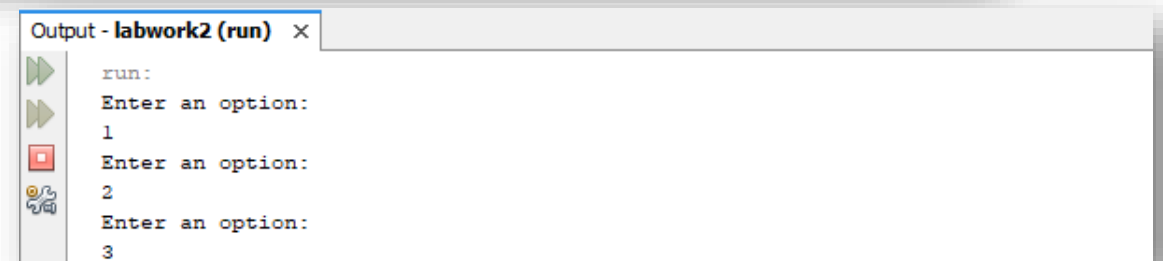
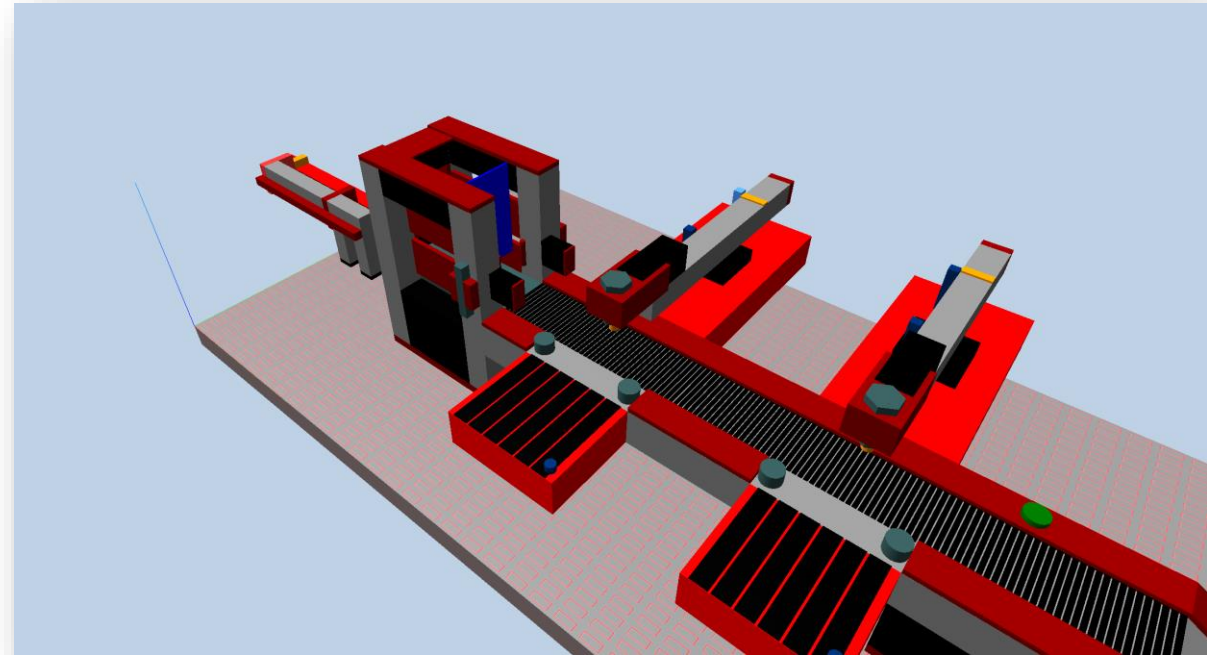
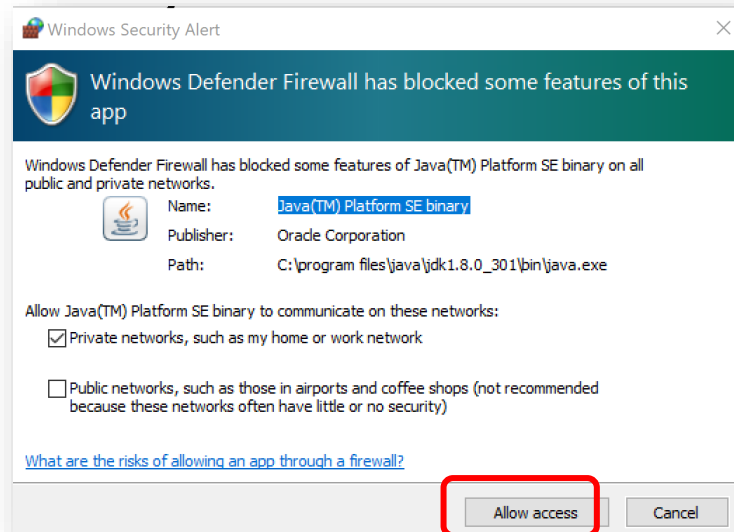
Add new Java class (2): Add a new class named **TestDemo.java**



```
13 public class TestDemo {
14
15     /**
16      * @param args the command line arguments
17      */
18     public static void main(String[] args) {
19         int t = -1;
20         SplitterConveyor.initializeHardwarePorts();
21         Cylinder1 cylinder1 = new Cylinder1();
22         while(t != 0){
23             System.out.println("Enter an option: ");
24             Scanner scan = new Scanner(System.in);
25             t = scan.nextInt();
26             scan.nextLine();
27             switch (t){
28                 case 1: cylinder1.moveBackward(); break;
29                 case 2: cylinder1.moveFoward(); break;
30                 case 3: cylinder1.stop(); break;
31             }
32         }
33     }
34
35 }
```

Testing from Java Application

Testing the demo: Right-click class TestDemo and select “Run File”, as illustrated in the picture



Testing from Java Application

If you get unsatisfied link error



Well, that's bad luck...

We need to repeat the process of using the [SplitterConveyor](#) through the JNI interface. Repeat the steps of this process very carefully.



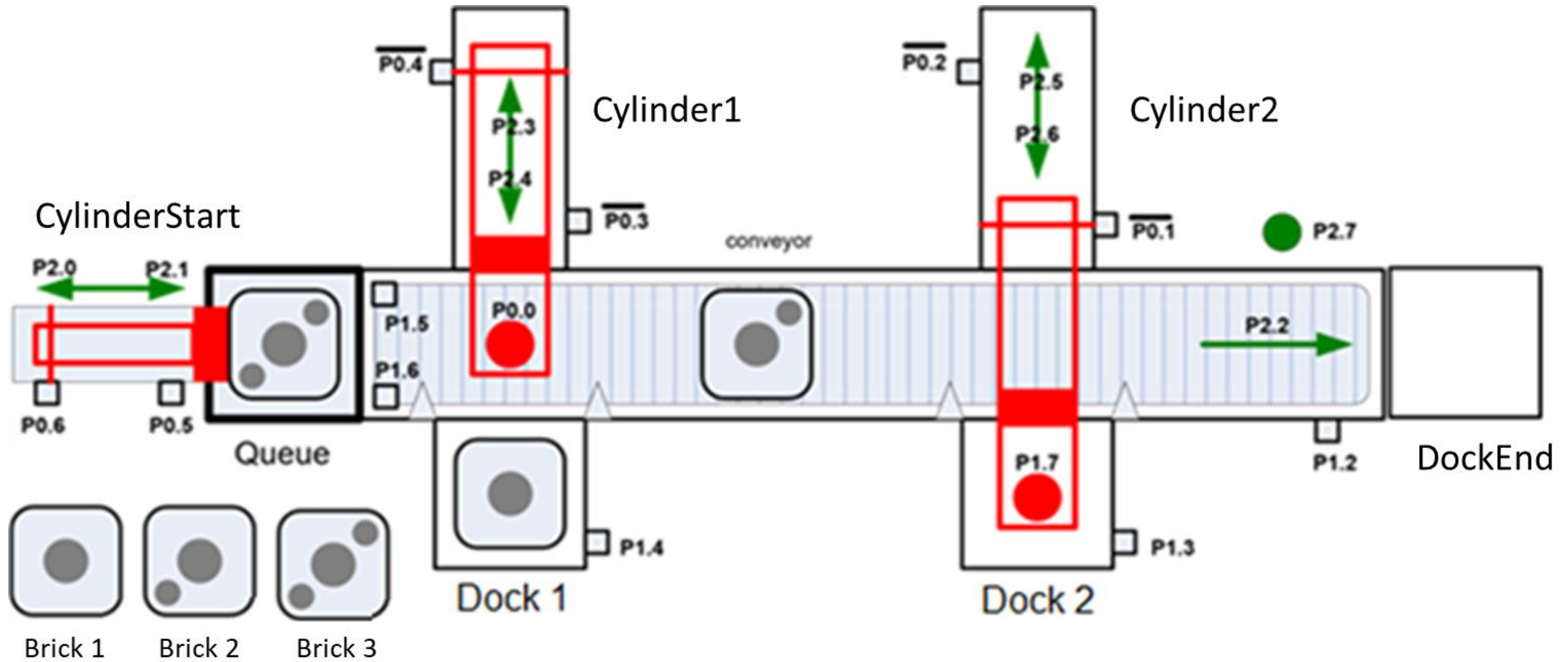
```
Output - labwork2 (run) x
Exception in thread "main" java.lang.UnsatisfiedLinkError: SplitterConveyor.cylinder1MoveForward()V
    at SplitterConveyor.cylinder1MoveForward(Native Method)
    at Cylinder1.moveForward(Cylinder1.java:12)
    at TestDemo.main(TestDemo.java:20)
C:\Users\ao\AppData\Local\NetBeans\Cache\12.5\executor-snippets\run.xml:111: The following error occurred while executing this line:
C:\Users\ao\AppData\Local\NetBeans\Cache\12.5\executor-snippets\run.xml:94: Java returned: 1
BUILD FAILED (total time: 11 seconds)
```

PART 2

Java Real-Time



LABWORK 2



The Cylinder Interface

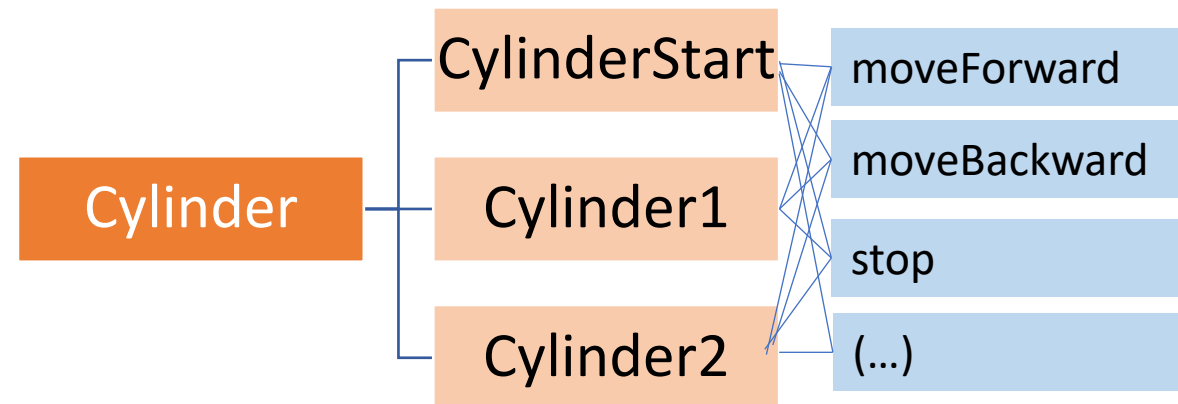
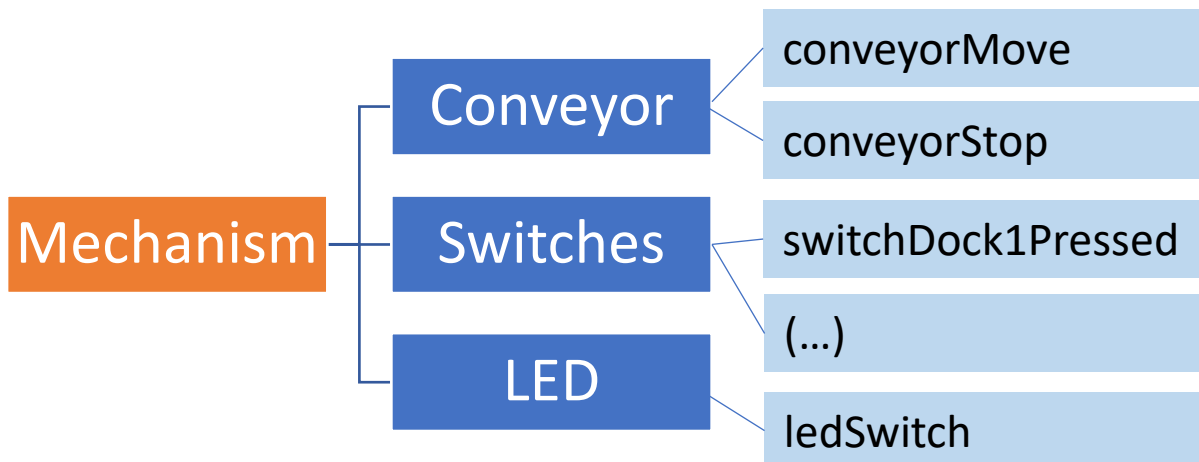
Sensors and actuators: the Java side of JNI



Having the class `SplitterConveyor.java` already developed (the DLL), at the Java side we need to organize our classes into a more logical and straightforward way.

This should be done by aggregating the splitter functionality as several well identified classes, some of them as Java threads.

For instance, we can define concrete classes for cylinders operation and a class named mechanism, which aggregates the functionality for the conveyor movement, the switches, and the LED.



The Cylinder Interface



Let's start by providing an interface definition for the Cylinders: Java interface -> **Cylinder.java**

```
9  /*
10 public interface Cylinder {
11     public void moveForward();
12     public void moveBackward();
13     public void stop();
14     public int getPosition(); //returns 0,1 or -1 (if undefined position)
15     public void gotoPosition(int position);
16     //(...)
17     public boolean brickDetected();
18 }
19
```

An **interface** is a completely "**abstract class**" that is used to group related methods with empty bodies

https://www.w3schools.com/java/java_interface.asp

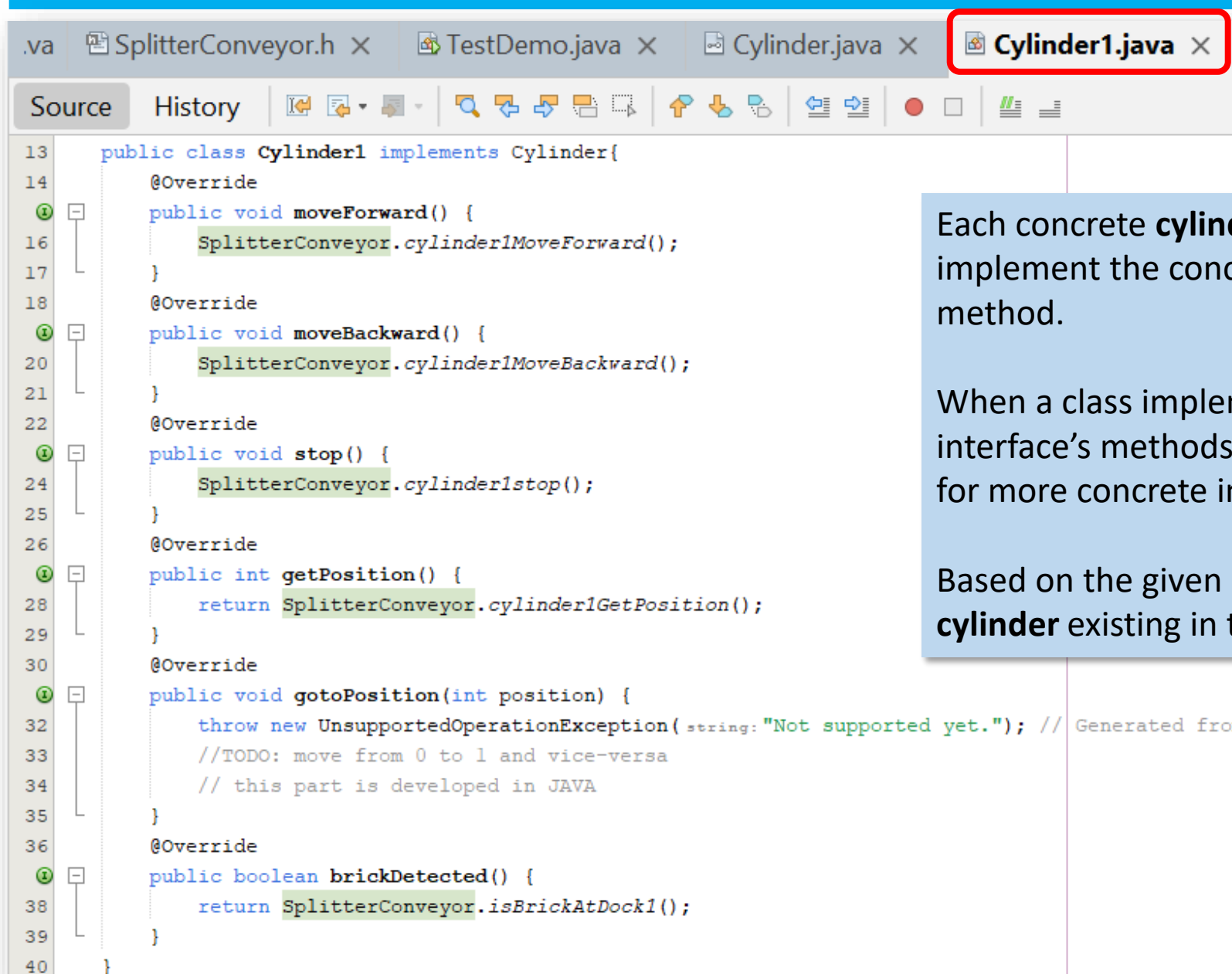
A class that implements a specific interface must implement the methods of the interface.

The interface has several methods that are used in each cylinder.

Although each cylinder has distinct implementation, e.g. cylinder start has "active_high" sensors, in an abstract way they behave similarly..... as such, interface **Cylinder** specifies and describes the behavior of each cylinder in an abstract way.

Some methods in the interface definition might not be needed, just to illustrate the concept.

The Cylinder Interface



```
13 public class Cylinder1 implements Cylinder{
14     @Override
15     public void moveForward() {
16         SplitterConveyor.cylinder1MoveForward();
17     }
18     @Override
19     public void moveBackward() {
20         SplitterConveyor.cylinder1MoveBackward();
21     }
22     @Override
23     public void stop() {
24         SplitterConveyor.cylinder1stop();
25     }
26     @Override
27     public int getPosition() {
28         return SplitterConveyor.cylinder1GetPosition();
29     }
30     @Override
31     public void gotoPosition(int position) {
32         throw new UnsupportedOperationException( string: "Not supported yet."); // Generated from
33         //TODO: move from 0 to 1 and vice-versa
34         // this part is developed in JAVA
35     }
36     @Override
37     public boolean brickDetected() {
38         return SplitterConveyor.isBrickAtDock1();
39     }
40 }
```

Each concrete **cylinder** can follow the **Cylinder interface** and implement the concrete functionality of each Cylinder's method.

When a class implements a cylinder, it must **implement all** the interface's methods (if not, see the messages from NetBeans for more concrete insight).

Based on the given interface, try develop **three classes** for each **cylinder** existing in the physical system.

Developing class Mechanism



Create a new class **Mechanism.java**

```
9  */
10 public class Mechanism {
11
12     public void conveyorMove() {
13         SplitterConveyor.conveyorMove();
14     }
15     public void ConveyorStop() {
16         SplitterConveyor.conveyorStop();
17     }
18     public boolean switchDock1Pressed() {
19         //TODO... for now return false
20         return false;
21     }
22     public boolean switchDock2Pressed() {
23         //TODO... for now return false
24         return false;
25     }
26     public boolean switchDockEndPressed() {
27         //TODO... for now return false
28         return false;
29     }
30     public void ledSwitch(int on) {
31         //TODO...
32     }
33 }
```

The class mechanism deals with:

- the conveyor
- the LED
- the three switches (one at each dock)

Remember, some of these methods might be static, others not

Static methods do not use any instance variables of any object of the class they are defined in. Static methods take all the data from parameters and compute something from those parameters, with no reference to variables.

<https://www.tutorialspoint.com/Java-static-method>

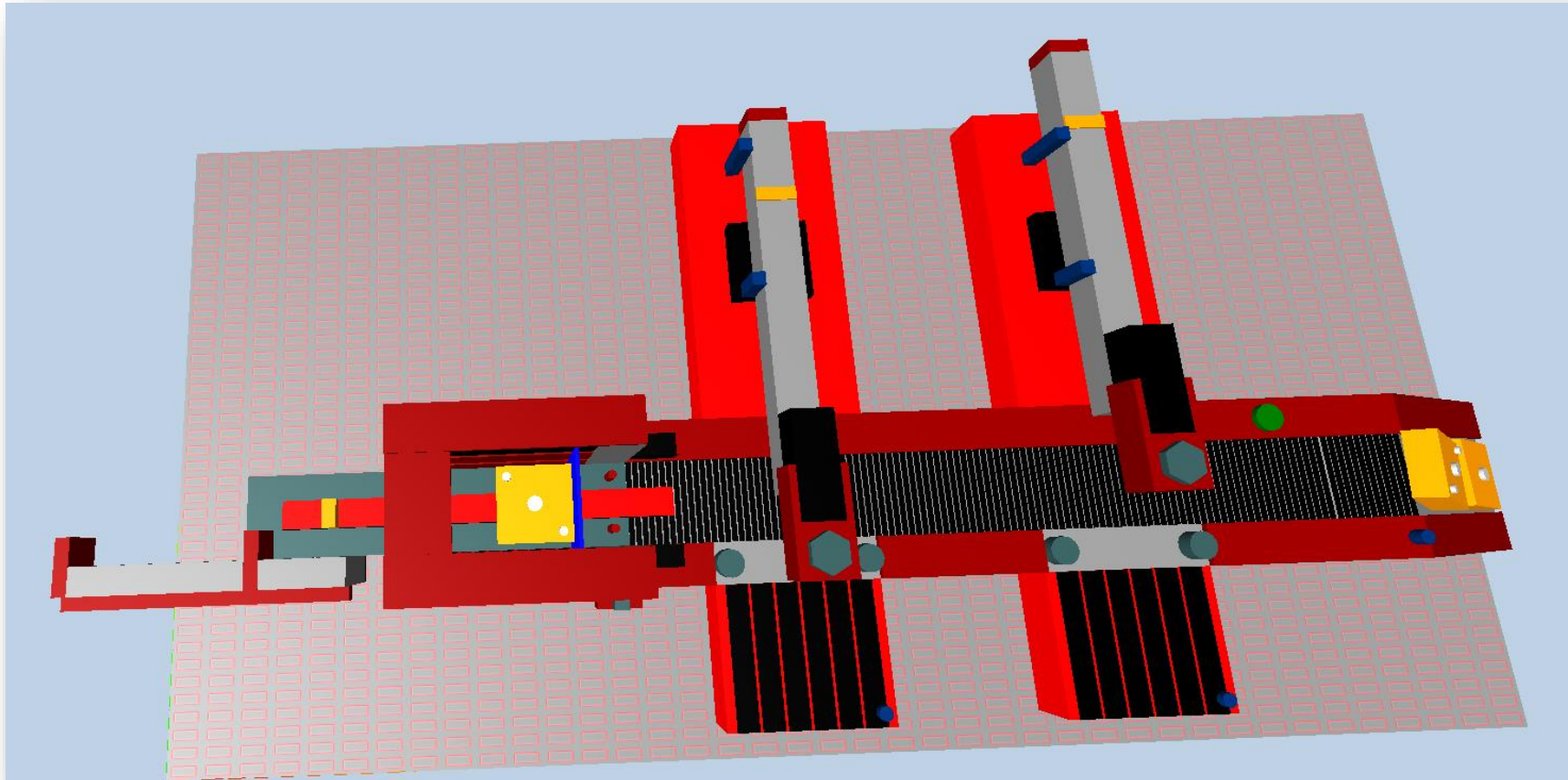
Identification of needed Java Threads

Aspects to consider

Check the Lab work 2 requirements and elaborate your work “architecture”....

.... which threads should be created to develop the necessary behaviors?

While some are sequential, others take place in parallel?

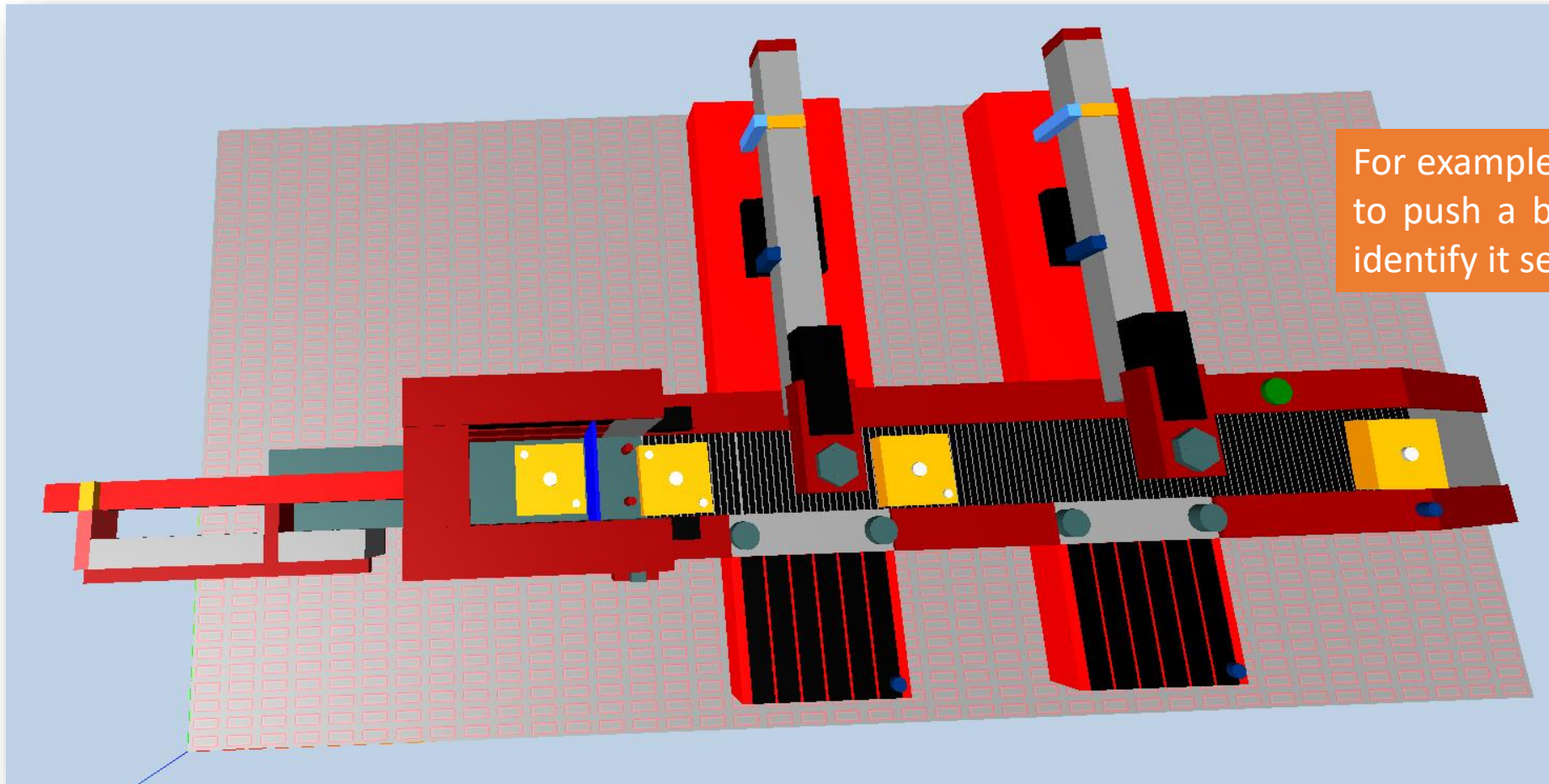


Identification of needed Java Threads

Aspects to consider

Identify which actions / behaviors can occur simultaneously in the **SplitterConveyor**

The question is how to orchestrate their execution...



For example, it is physically not possible to push a brick with cylinder start and identify it sequentially.

To study....



Check....

<https://docs.oracle.com/javase/8/docs/api/index.html>

PREV CLASS NEXT CLASS FRAMES NO FRAMES
SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3
java.lang.
Class Thread
java.lang.Thread

All Implemented Interfaces:
Runnable

Direct Known Subclasses:
ForkJoinWorkerThread

public class Thread
extends Object
implements Runnable

A *thread* is a thread of execution in a program. The Java Virtual Machine allows an application to have multiple threads of execution running concurrently.

Every thread has a priority. Threads with higher priority are executed in preference to threads with lower priority. Each thread may or may not also be marked as a daemon. When code running in some thread creates a new Thread object, the new thread has its priority initially set equal to the priority of the creating thread, and is a daemon thread if and only if the creating thread is a daemon.

When a Java Virtual Machine starts up, there is usually a single non-daemon thread (which typically calls the method name of some designated class). The Java Virtual Machine continues to execute threads until either of the following occurs:

- The exit method of class Runtime has been called and the security manager has permitted the exit operation to take place.
- All threads that are not daemon threads have died, either by returning from the call to the run method or by throwing an exception that propagates beyond the run method.

There are two ways to create a new thread of execution. One is to declare a class to be a subclass of Thread. This subclass must override the run method of class Thread. An instance of the subclass can then be allocated and started. For example, a thread that computes primes larger than a stated value could be written as follows:

```
class PrimeThread extends Thread {  
    long minPrime;  
    PrimeThread(long minPrime) {  
        this.minPrime = minPrime;  
    }  
}
```

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP
PREV PACKAGE NEXT PACKAGE FRAMES NO FRAMES

Package java.util.concurrent
Utility classes commonly useful in concurrent programming.
See: Description

Interface Summary

Interface	Description
BlockingDeque<E>	A Deque that additionally supports blocking operations that wait for the deque to become non-empty when retrieving an element, and wait for space to become available in the deque when storing an element.
BlockingQueue<E>	A Queue that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element.
Callable<V>	A task that returns a result and may throw an exception.
CompletableFuture.AsyncronousCompletionTask	A marker interface identifying asynchronous tasks produced by async methods.
CompletionService<V>	A service that decouples the production of new asynchronous tasks from the consumption of the results of completed tasks.
CompletionStage<T>	A stage of a possibly asynchronous computation, that performs an action on computes a value when another CompletionStage completes.
ConcurrentMap<K,V>	A Map providing thread safety and atomicity guarantees.
ConcurrentNavigableMap<K,V>	A ConcurrentMap supporting NavigableMap operations, and recursively so for its navigable sub-maps.
Delayed	A mix-in style interface for marking objects that should be acted upon after a given delay.
Executor	An object that executes submitted Runnable tasks.
ExecutorService	An Executor that provides methods to manage termination and methods that can produce a Future for tracking progress of one or more asynchronous tasks.
ForkJoinPool.ForkJoinWorkerThreadFactory	Factory for creating new ForkJoinWorkerThreads .
ForkJoinPool.ManagedBlocker	Interface for extending managed parallelism for tasks running in ForkJoinPools .

<https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html>



KEEP UP THE
GOOD
WORK!